

Laravel Studio

A quieter place to build Laravel. Feature, setup and platform guide for the desktop studio.

Version 1.0.1

WINDOWS · LINUX · MACOS

Dark by design.
Built on Eclipse Theia.

01 1. Platform and installation
02 2. Editor and workbench
03 3. Laravel project dashboard
04 4. PHP and Laravel language tooling
05 5. Debugging
06 6. Terminal
07 7. Remote development
08 8. VS Code extension support
09 9. Design and comfort
10 10. Performance and security
11 11. Commands, menus and shortcuts
12 12. Packaging and deployment
13 13. Known limitations
14 14. Verification performed

Laravel Studio is a free, cross-platform, dark-only desktop IDE for Laravel. It combines a focused JetBrains-inspired workbench with a real VS Code extension host, Theia's editor, terminal, Git, debugger and remote infrastructure, and a Laravel-specific workspace dashboard.

1. Platform and installation

- Windows 10/11 x64 installer (NSIS) and portable executable.
- Linux x64 packages: AppImage and Debian package, built on Linux through the included GitHub Actions workflow.
- macOS packages: DMG and ZIP for Intel and Apple Silicon, built on macOS through the included workflow.
- Native Electron application; no browser required.
- Dark-only visual design. No light theme is selected on first run.
- Direct-to-workbench startup with the file explorer available when no project is open.
- Automatic project detection and a remembered per-project layout.
- Windows CA certificate integration for proxy-aware networking.
- Workspace Trust, so a downloaded Laravel project is not executed before permission is granted.

2. Editor and workbench

- Monaco-based code editor with syntax highlighting, code folding, multi-cursor editing, find and replace, bracket matching, indentation guides, and smooth caret motion.
- Bundled JetBrains Mono editor font with discretionary ligatures, and Inter for the UI.
- Project explorer with a virtualized file tree, open editors, file operations, and an active file status.
- Seti file and folder icons, folder selection on workspace open, and automatic reveal of the active editor file in the explorer.
- Tabs, split editors, breadcrumb support (disabled in the default minimal layout), and editor groups.
- Full-text search across the project and replace support.
- Quick Open for files and a command palette for every command.
- Outline, Problems, Debug Console, Output, Terminal, and Testing views.
- Integrated Git source control through the VS Code Git extension and Theia SCM.
- Git status decorations, diffs, merge conflict helpers, and a file-level change overview.

- Settings editor and JSON settings files.
- Keyboard-driven navigation and visible focus styles.
- Resizable panel layout with saved positions per project.
- Project-aware status bar showing the active Laravel version.

3. Laravel project dashboard

Open it from the Laravel activity rail, the Laravel menu, the command palette, or the Laravel status bar item.

Overview

- Laravel version, PHP version, application environment, environment URL, database driver, test runner, Git branch and dirty state.
- One-click Start server, Run tests, Routes, Database, Artisan and Containers actions.
- Quick maintenance actions: migrate, clear caches, Composer install, debugger setup, and a guarded fresh-migration command.
- Project health diagnostics for `.env`, `APP_KEY`, Composer dependencies, writable `storage/` and `bootstrap/cache/`, database configuration, PHP runtime, Xdebug and frontend assets.
- Toolchain status for PHP, Composer, Git, Docker and Xdebug.

Routes

- Parses `php artisan route:list --json` from the project.
- Search by method, URI, route name, controller, or middleware.
- Colour-coded HTTP method badges.
- Shows controller actions and middleware layer counts.
- Opens controller files directly in the editor when the action is a class method.
- Handles closures and framework routes such as Laravel's `/up` health endpoint.

Artisan

- Loads every registered Artisan command, grouped by namespace.
- Search across command names and descriptions.
- Run a command and capture its output inside the dashboard.
- Send a command to the real integrated terminal when a prompt, long process, or TTY is required.
- Background-safe execution with `--no-interaction` handled as a global Artisan option.
- Shows exit status and duration, with clear stdout/stderr separation.

Tests

- Detects Pest, PHPUnit, or Artisan's test runner.
- Run the full suite or filter by test name/class/method.
- Shows pass/fail status, duration, and captured test output.
- Parses JSON test summaries when the runner provides them.
- A terminal runner is available for interactive test runs.

Database

- Reads database settings from `.env` in the backend; passwords are never exposed to the renderer.
- SQLite support using the Node built-in SQLite module.
- MySQL/MariaDB support using mysql2.
- PostgreSQL support using pg.
- Table list with filtering.
- One-click table preview.

- SQL editor with Ctrl/Cmd+Enter execution.
- Row count, execution time, affected-row messages and error reporting.
- Result browser with NULL, boolean and numeric value treatment.
- Results capped at 500 rows for responsiveness.
- Clear connection errors for refused connections, invalid credentials and missing schemas.

Containers

- Detects Docker CLI, Docker Compose and the Docker engine.
- Detects Laravel Sail projects.
- Detects `compose.yaml`, `compose.yml`, `docker-compose.yml`, and `docker-compose.yaml`.
- Shows services, state, container name and published ports.
- Start, stop, restart, build and inspect a Compose stack.
- Follow logs in the integrated terminal.
- Captured recent logs per service.
- Clear guidance when Docker is missing or the engine is stopped.

4. PHP and Laravel language tooling

- Official Laravel VS Code extension and Laravel Language Server, with Laravel-aware completions, diagnostics, links and code actions. The extension can download and manage its LSP binary.
- PHP Intelephense for PHP completions, navigation and workspace indexing.
- PHP Debug (`xdebug.php-debug`) for Xdebug debugging.
- Laravel Blade language support and snippets.
- Laravel 5 snippets for routes, controllers, models, migrations, Eloquent and validation.
- Tailwind CSS IntelliSense.
- EditorConfig support.
- DotENV highlighting.
- Built-in language support retained for PHP, JavaScript, TypeScript, HTML, CSS, SCSS, Less, JSON, Markdown, YAML, SQL, shell, PowerShell and Dockerfiles.
- Language grammars and snippets load only for the languages included in the build.

5. Debugging

- One-click creation of a project `.vscode/launch.json` PHP debug configuration.
- Listens on the standard Xdebug port `9003`.
- Detects whether Xdebug is loaded and reports its version, mode, client host and client port.
- Integrates with Theia's Debug view and the bundled PHP Debug adapter.
- Existing launch configurations are preserved; comments or invalid JSON are never overwritten.

6. Terminal

- Real integrated terminal with xterm.js and native pseudo-terminal support.
- Working-directory-aware project terminal with a one-click launcher.
- Artisan, Composer, test, migration and long-running commands can be sent to the real terminal.
- Windows ConPTY support.
- Terminal tabs, search, copy/paste, shell integration and command history features from Theia.
- Terminal output is never routed through a shell string constructed from untrusted input; dashboard commands use argument arrays.

7. Remote development

- Theia's native remote connection layer.
- `Laravel: Connect to Remote Host over SSH…` command.
- SSH config hosts, key authentication, passphrase support, agent support and port forwarding through Theia.
- Remote file system and terminal are served by the remote backend, not by a local-only tunnel.
- WSL remote support on Windows through Theia Remote WSL.
- A remote side needs SSH access and permission for Theia to install its small backend and Node runtime.

8. VS Code extension support

- Real VS Code extension host provided by Theia.
- Open VSX registry built in.
- Extensions view for search, install, disable, uninstall and update.
- Local `.vsix` installation.
- Bundled extensions can be enabled or disabled like regular extensions.
- Extension settings are available in the normal settings editor.
- Full support depends on the extension's use of VS Code APIs. Extensions that require the Microsoft Marketplace, closed Microsoft services, or proprietary remote protocols may not run. Open VSX and VSIX installation are the supported paths.

9. Design and comfort

- Dark-only, graphite palette with muted blue focus, green success, amber warning and coral error states.
- JetBrains-inspired layout: project panel, navigator, editor tabs, optional inspector, and bottom panel.
- Bundled Inter and JetBrains Mono variable fonts.
- Custom Laravel dashboard built as a native workbench view.
- High-DPI-ready vector iconography with no emoji-as-icon shortcuts.
- Visible keyboard focus, labelled controls, keyboard-operable tabs, semantic tables and ARIA status announcements.
- Reduced-motion support.
- Subtle 120–180 ms interactions rather than decorative animation.
- Responsive custom views with a collapsed-navigation fallback for narrow sidebars.
- Resizable panes and remembered split sizes.
- First-run sidebar widened and made scrollable; health labels now shrink cleanly rather than being clipped at narrower panel sizes.

10. Performance and security

- Native Theia infrastructure keeps language servers and extensions off the renderer's critical path.
- Project metadata is cached briefly; routes and Artisan listings have explicit refresh actions.
- Language extensions were trimmed to Laravel's actual workflow, reducing plugin startup work compared with the full Theia extension set.
- Database results are limited and output is bounded.
- File watchers and search exclude `vendor/`, `node_modules/`, storage framework caches and frontend build output.
- All filesystem and process operations run in the Node backend.
- Commands execute without shell interpolation; SQL is passed to the configured driver.

- `.env` credentials never leave the backend and are never included in project metadata returned to the renderer.
- Workspace Trust is supported, although the test harness can disable it explicitly.

11. Commands, menus and shortcuts

All actions are available from the command palette. The top-level **Laravel** menu includes:

- Open Project Dashboard
- Open Project Folder…
- New Laravel Project…
- Start Development Server
- Run Test Suite
- Browse Routes
- Browse Artisan Commands
- Open Database
- Open Containers
- Run Migrations
- Fresh Migrations… (explicit confirmation)
- Clear Caches
- Set Up PHP Debugging
- Connect to Remote Host over SSH…

Native workbench shortcuts such as Quick Open, Command Palette, Find, Find in Files, toggle terminal, save, and standard editor navigation remain available through Theia.

12. Packaging and deployment

- Windows NSIS installer with install-directory selection and optional launch.
- Windows portable executable.
- Linux AppImage and `.deb` targets, built natively in CI.
- macOS DMG and ZIP targets, built natively in CI.
- GitHub Actions matrix for Windows, Ubuntu and macOS.
- Unpacked preview output for validation.
- App icon set generated for Windows, macOS and Linux.
- Custom splash screen and product branding.

13. Known limitations

- Laravel Studio is not Microsoft VS Code and does not include Microsoft-exclusive marketplace extensions or services.
- Some extensions depend on private VS Code APIs and may fail or degrade. The extension manager surfaces installation and activation errors in its output.
- Xdebug must be installed and configured in the PHP runtime; Laravel Studio detects but does not edit `php.ini`.
- Docker actions require a running Docker engine. Docker Desktop is not bundled.
- Remote SSH requires a reachable host and permission to install Theia's remote backend.
- MySQL/PostgreSQL integration requires those database servers to be running.
- The Windows build is unsigned until production signing credentials are supplied; Windows SmartScreen may warn on first launch.

- Linux and macOS binaries need to be produced on their native CI runners because native terminal watcher/keychain modules are platform-specific.

14. Verification performed

- TypeScript compilation of the Laravel Studio extension.
- Unit tests for `.env` parsing, JSON extraction, Composer lock parsing, route/Artisan parsing, command tokenization and SQLite path handling.
- Backend integration test against a fixture Laravel project covering project inspection, routes, Artisan execution, test execution, SQLite schema/query execution, debug launch setup and Docker detection.
- Live run against a real Laravel 13 project created with Composer.
- Live validation of routes, Artisan output, database queries, dashboard tabs, project health, the VS Code extension host, Open VSX extension loading, the custom theme, and a real integrated terminal command.